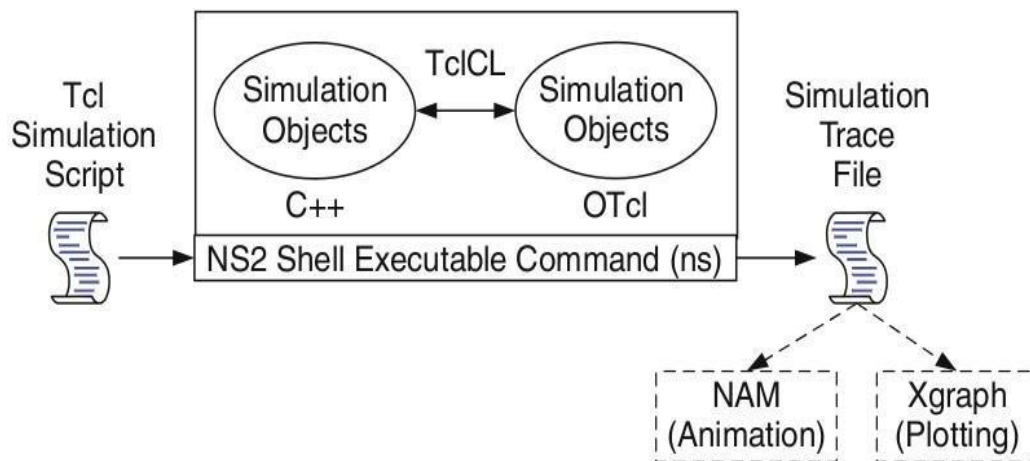


INTRODUCTION TO NS-2

- Widely known as NS2, is simply an event driven simulation tool.
- Useful in studying the dynamic nature of communication networks.
- Simulation of wired as well as wireless network functions and protocols (e.g., routing algorithms, TCP, UDP) can be done using NS2.
- In general, NS2 provides users with a way of specifying such network protocols and simulating their corresponding behaviors.

Basic Architecture of NS2



Tcl scripting

- Tcl is a general purpose scripting language.[Interpreter]
- Tcl runs on most of the platforms such as Unix, Windows, and Mac.
- The strength of Tcl is its simplicity.
- It is not necessary to declare a data type for variable prior to the usage.

Basics of TCL

Syntax: command arg1 arg2 arg3

- **Hello World!**

```
puts stdout{Hello, World!} Hello, World!
```

- **Variables** CommandSubstitution

```
set a5    set len [string length foobar]
```

```
set b$a    set len [expr [string length foobar] +9]
```

Loops

```
while{$i < $n} {      for {set i 0} {$i < $n} {incr i}{  
  }  
}
```

Wired TCL Script Components

- Create the event scheduler
- Open new files & turn on the tracing
- Create the nodes
- Setup the links
- Configure the traffic type (e.g., TCP, UDP, etc)
- Set the time of traffic generation (e.g., CBR, FTP)
- Terminate the simulation

NS Simulator Preliminaries.

1. Initialization and termination aspects of the ns simulator.
2. Definition of network nodes, links, queues and topology.
3. Definition of agents and of applications.
4. The nam visualization tool.
5. Tracing and random variables.

Initialization and Termination of TCL Script in NS-2

An ns simulation starts with the command

set ns [new Simulator]

Which is thus the first line in the tcl script? This line declares a new variable as using the set command, you can call this variable as you wish, In general people declares it as ns because it is an instance of the Simulator class, so an object the code[new Simulator] is indeed the installation of the class Simulator using the reserved word new.

In order to have output files with data on the simulation (trace files) or files used for visualization (nam files), we need to create the files using “open” command:

#Open the Trace file

set tracefile1 [open out.tr w]

#Open the NAM trace file

\$ns trace-all \$tracefile1

The above creates a data trace file called “out.tr” and a nam visualization trace file called “out.nam”. Within the tcl script, these files are not called explicitly by their names, but instead by pointers that are declared above and called “tracefile1” and “namfile” respectively. Remark that they begins with a # symbol. The second line open the file “out.tr” to be used for writing, declared with the letter “w”. The third line uses a simulator method called trace-all that have as parameter the name of the file where the traces will go.

The last line tells the simulator to record all simulation traces in NAM input format. It also gives the file name that the trace will be written to later by the command \$ns flush-trace. In our case, this will be the file pointed at by the pointer “\$namfile”, i.e the file “out.tr”.

The termination of the program is done using a “finish” procedure.

#Define a ‘finish’ procedure

```
Proc finish { } {  
    global ns tracefile1 namfile  
    $ns flush-trace  
    Close  
    $tracefile1  
    Close $namfile
```

The word proc declares a procedure in this case called **finish** and without arguments. The word **global** is used to tell that we are using variables declared outside the procedure. The simulator method “**flush-trace**” will dump the traces on the respective files. The tcl command “**close**” closes the trace files defined before and **exec** executes the nam program for visualization. The command **exit** will ends the application and return the number 0 as status to the system. Zero is the default for a clean exit. Other values can be used to say that is a exit because something fails.

At the end of ns program we should call the procedure “finish” and specify at what time the termination

```
$ns at 125.0 “finish”
```

should occur. For example,

will be used to call “**finish**” at time 125sec. Indeed, the **at** method of the simulator allows us to schedule events explicitly.

The simulation can then begin using the command

Definition of a network of links and nodes

The way to define a node is

```
set n0 [$ns node]
```

We created a node that is printed by the variable n0. When we shall refer to that node in the script we shall thus write \$n0.

Once we define several nodes, we can define the links that connect them. An example of a definition of a link is:

\$ns duplex-link \$n0 \$n2 10Mb 10ms DropTail

Which means that \$n0 and \$n2 are connected using a bi-directional link that has 10ms of propagation delay and a capacity of 10Mb per sec for each direction.

To define a directional link instead of a bi-directional one, we should replace “duplex- link” by “simplex-link”.

In NS, an output queue of a node is implemented as a part of each link whose input is that node. The definition of the link then includes the way to handle overflow at that queue. In our case, if the buffer capacity of the output queue is exceeded then the last packet to arrive is dropped. Many alternative options exist, such as the RED (Random Early Discard) mechanism, the FQ (Fair Queuing), the DRR (Deficit Round Robin), the stochastic Fair Queuing (SFQ) and the CBQ (which including a priority and a round-robin scheduler).

In ns, an output queue of a node is implemented as a part of each link whose input is that node. We should also define the buffer capacity of the queue related to each link. An example would be:

#set Queue Size of link (n0-n2) to 20
\$ns queue-limit \$n0 \$n2 20

Agents and Applications

We need to define routing (sources, destinations) the agents (protocols)the application that use them.

FTP over TCP

TCP is a dynamic reliable congestion control protocol. It uses Acknowledgements created by the destination to know whether packets are well received.

There are number variants of the TCP protocol,such as Tahoe,Reno, New Reno, Vegas.

The type of agent appears in the first line:

set tcp [new Agent/TCP]

The command **\$ns attach-agent \$n0 \$tcp** defines the source node of the tcp connection.

The command

Defines the behaviour of the destination node of TCP and assigns to it a pointer called sink.

#Setup a UDP connection

```
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n5 $null
$ns connect $udp $null
$udp set fid_2
```

#setup a CBR over UDP connection

```
set cbr [new
Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set packetSize_ 100
$cbr set rate_ 0.01Mb
```

Above shows the definition of a CBR application using a UDP agent

The command **\$ns attach-agent \$n4 \$sink** defines the destination node. The command **\$ns connect \$tcp \$sink** finally makes the TCP connection between the source and destination nodes.

TCP has many parameters with initial fixed defaults values that can be changed if mentioned explicitly. For example, the default TCP packet size has a size of 1000bytes. This can be changed to another value, say 552bytes, using the command **\$tcp set packetSize_ 552**.

When we have several flows, we may wish to distinguish them so that we can identify them with different colors in the visualization part. This is done by the command **\$tcp set fid_ 1** that assigns to the TCP connection a flow identification of “1”. We shall later give the flow identification of “2” to the UDP connection.

CBR over UDP

A UDP source and destination is defined in a similar way as in the case of TCP.

Instead of defining the rate in the command `$cbr set rate_ 0.01Mb`, one can define the time interval between transmission of packets using the command.

`$cbr set interval_ 0.005`

The packet size can be set to some value using

`$cbr set packetSize_ <packet size>`

Scheduling Events

NS is a discrete event based simulation. The `tcpscript` defines when events should occur. The initializing command `set ns [new Simulator]` creates an event scheduler, and events are then scheduled using the format:

`$ns at <time><event>`

The scheduler is started when running `ns` that is through the command `$ns run`.

The beginning and end of the FTP and CBR application can be done through the following command

`$ns at 0.1 "$cbr start"`

`$ns at 1.0 "$ftp start"`

`$ns at 124.0 "$ftp stop"`

Structure of Trace Files

When tracing into an output ASCII file, the trace is organized in 12 fields as follows in fig shown below, The meaning of the fields are:

Event	Time	From Node	To Node	PKT	PKT	Flags	Fid	Src Addr	Dest Addr	Seq Num	Pkt id
-------	------	-----------	---------	-----	-----	-------	-----	----------	-----------	---------	--------

1. The first field is the event type. It is given by one of four possible symbols `r`, `+`, `-`, `d` which correspond respectively to receive (at the output of the link), enqueued, dequeued and dropped.
2. The second field gives the time at which the event occurs.
3. Gives the input node of the link at which the event occurs.
4. Gives the output node of the link at which the event occurs.
5. Gives the packet type (eg CBR or TCP)
6. Gives the packet size
7. Some flags

8. This is the flow id(fid) of IPv6 that a user can set for each flow at the input OTcl script one can further use this field for analysis purposes; it is also used when specifying stream color for the NAM display.
9. This is the source address given in the form of “node.port”.
10. This is the destination address, given in the same form.
11. This is the network layer protocol’s packet sequence number. Even though UDP implementations in a real network do not use sequence number, ns keeps track of UDP packet sequence number for analysis purposes
12. The last field shows the unique id of the packet.

XGRAPH

The xgraph program draw sagraph on an x-display given data read from either data file or from standard input if no files are specified. It can display upto 64 independent data sets using different colors and line styles for each set. It an not a test hegraph with a title, axis labels, grid lines or tick marks, grid labels and aleg end.

Syntax:

Xgraph [options] file-name

Options are listed here

`/-bd <color>` (Border)

This specifies the border color of the xgraph window.

`/-bg <color>` (Background)

This specifies the background color of the xgraph window.

`/-fg<color>` (Foreground)

This specifies the foreground color of the xgraph window.

`/-lf <fontname>` (LabelFont)

All axis labels and grid labels are drawn using this font.

`/-t<string>` (Title Text)

This string is centered at the top of the graph.

`/-x <unit name>` (XunitText)

This is the unit name for the x-axis. Its default is “X”.

`/-y <unit name>` (YunitText)

This is the unit name for the y-axis. Its default is “Y”.

Awk- An Advanced

awk is a programmable, pattern-matching, and processing tool available in UNIX. It works equally well with text and numbers.

awk is not just a command, but a programming language too. In other words, awk utility is a pattern scanning and processing language. It searches one or more files to see if they contain lines that match specified patterns and then perform associated actions, such as writing the line to the standard output or incrementing a counter each time it finds a match.

Syntax:

awk option 'selection_criteria {action}' file(s)

Here, selection_criteria filters input and select lines for the action component to act upon. The selection_criteria is enclosed within single quotes and the action within the curly braces. Both the selection_criteria and action forms an awk program.

Example: \$ awk '/manager/ {print}' emp.lst Variables

Awk allowstheusertousevariables of here choice. You can now print a serial number, using the variable kount, and apply it those directors drawing a salary exceeding 6700:

```
$ awk -F"|" ' $3 == "director" && $6 > 6700 { count =count+1
```

```
printf " %3f %20s %-12s %d\n", count,$2,$3,$6 }' empn.lst
```

THE -f OPTION: STORING awk PROGRAMS IN A FILE

You should hold large awk programs in separate file and provide them with the awk extension for easier identification. Let's first store the previous program in the file empawk.awk:

```
$ cat empawk.awk
```

Observe that this time we haven't used quotes to enclose the awk program. You can now use awk with the -f *filename* option to obtain the same output:

Awk -F"|" -f empawk.awk empn.lst

THE BEGIN AND END SECTIONS

Awk statements are usually applied to all lines selected by the address, and if there are no addresses, then they are applied to every line of input. But, if you have to print something before processing the first line, for example, a heading, then the BEGIN section can be used gainfully. Similarly, the end section useful in printing some totals after processing is over.

The BEGIN and END sections are optional and take the form

BEGIN {action} END {action}

These two sections, when present, are delimited by the body of the awk program. You can use them to print a suitable heading at the beginning and the average salary at the end.

BUILT-IN VARIABLES

Awk has several built-in variables. They are all assigned automatically, though it is also possible for a user to reassign some of them. You have already used NR, which signifies the record number of the current line. We'll now have a brief look at some of the other variables. **The FS Variable:** as stated elsewhere, awk uses a contiguous string of spaces as the default field delimiter. FS redefines this field separator, which in the sample database happens to be the |. When used at all, it must occur in the BEGIN section so that the body of the program knows its value before it starts processing:

BEGIN {FS="|"} }

This is an alternative to the -F option which does the same thing.

The OFS Variable: when you used the print statement with comma-separated arguments, each argument was separated from the other by a space. This is awk's default output field separator, and can be reassigned using the variable OFS in the BEGIN section:

BEGIN { OFS="~" } }

When you reassign this variable with a ~ (tilde), awk will use this character for delimiting the print arguments. This is a useful variable for creating lines with delimited fields.

The NF variable: NF comes in quite handy for cleaning up a database of lines that don't contain the right number of fields. By using it on a file, say emp.lst, you can locate those lines not having 6 fields, and which have crept in due to faulty data entry:

\$awk 'BEGIN {FS = "|"} NF != 6 {

Print "Record No ", NR, "has", "fields"}' emp.lst

The FILENAME Variable: FILENAME stores the name of the current file being processed. Like grep and sed, awk can also handle multiple filenames in the command line. By default, awk doesn't print the filename, but you can instruct it to do so:

'\$6<4000 {print FILENAME, \$0 }'

With FILENAME, you can devise logic that does different things depending on the file that is processed.

NS2 Installation

- NS2 is a free simulation tool.
- It runs on various platforms including UNIX (or Linux), Windows, and Mac systems.
- NS2 source codes are distributed in two forms: the all-in-one suite and the component-wise.
- ‘all-in-one’ package provides an “install” script which configures the NS2 environment and creates NS2 executable file using the “make” utility.

NS-2 installation steps in Linux

- Go to **Computer • File System** • now paste the zip file “**ns-allinone-2.34.tar.gz**” into opt folder.
- Now **unzip** the file by typing the following **command**

```
[root@localhost opt] # tar -xvzfns-allinone-2.34.tar.gz
```

- After the files get extracted, we get ns-allinone-2.34 folder as well as zip file ns-allinone-2.34.tar.gz

```
[root@localhost opt]#ns-allinone-2.34      ns-allinone-2.34.tar.gz
```

- Now go to ns-allinone-2.33 folder and install it [root@localhost opt] # **cd ns-allinone-2.34**
[root@localhost ns-allinone-2.33] #./install
- Once the installation is completed successfully we get certain pathnames in that terminal which must be pasted in “**.bash_profile**” file.
- First**minimizetheterminal**whereinstallationisdoneand**openanewterminal**andopen the file“**.bash_profile**”

```
[root@localhost ~] # vi.bash_profile
```

- When we open this file, we get a line in that file which is shown below

PATH=\$PATH:\$HOME/bin

To this line we must paste the path which is present in the previous terminal where **ns** was **installed**. First put “**:**” then paste the path in-front of bin. That path is shown below. “**:/opt/ns-allinone-2.33/bin:/opt/ns-allinone-2.33/tcl8.4.18/unix:/opt/ns-allinone- 2.33/tk8.4.18/unix**”.

- In the next line type “**LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:**” and paste the **two paths** separated by “**:**” which are present in the previous terminal i.e **Important notices section(1)**

“/opt/ns-allinone-2.33/otcl-1.13:/opt/ns-allinone-2.33/lib”

- In the next line type “**TCL_LIBRARY=\$TCL_LIBRARY:**” and paste the path which is present in previous terminal i.e **Important Notices section(2)**

“/opt/ns-allinone-2.33/tcl8.4.18/library”

- In the next line type “**exportLD_LIBRARY_PATH**”
- In the next line type “**exportTCL_LIBRARY**”
- The next two lines are already present the file “**export PATH**” and “**unsetUSERNAME**”
- **Save the program (ESC + shift : wq and press enter)**
- Now in the terminal where we have opened **.bash_profile** file, type the following command to **check if path is updated correctly or not**

```
[root@localhost ~] # vi.bash_profile
```

```
[root@localhost ~] # source.bash_profile
```

- If **path is updated properly**, then we will **get the prompt** as shown below
[root@localhost ~]#

- Now open the previous terminal where you have installed **ns**

```
[root@localhost ns-allinone-2.33] #
```

- Here we need to configure three packages “**ns-2.33**”, “**nam-1.13**” and “**xgraph-12.1**”
- **First**, configure “**ns-2.33**” package as shown below
[root@localhost ns-allinone-2.33] # **cd ns-2.33**
[root@localhost ns-2.33] # **./configure**
[root@localhost ns-2.33] # **make**
[root@localhost ns-2.33] # **make install**
[root@localhost ns-2.33] #**ns**

%

- If we get “**%**” symbol it indicates that **ns-2.33 configuration** was **successful**.

```
[root@localhost ns-allinone-2.33] # cd xgraph-12.1 [root@localhost xgraph-12.1] #  
./configure [root@localhost xgraph-12.1] # make clean [root@localhost xgraph-12.1] # make  
[root@localhost xgraph-12.1] # make install [root@localhost xgraph-12.1] #ns%
```

This completes the installation process of “NS-2” simulator.

1. Implement Three nodes point-to-point network with duplex links between them for different topologies. Set the queue size, vary the bandwidth, and find the number of packets dropped for various iterations.

```
import java.util.Scanner;

class Queue {
    int[] q;
    int f = 0, r = 0, size;

    public Queue(int size) {
        this.size = size;
        q = new int[size];
    }

    public void insert(int n) {
        Scanner in = new Scanner(System.in);
        for (int i = 0; i < n; i++) {
            System.out.print("\nEnter element " + (i + 1) + ": ");
            int ele = in.nextInt();
            if (r >= size) {
                System.out.println("\nQueue is full. Lost Packet: " + ele);
                break;
            } else {
                q[r++] = ele;
            }
        }
    }

    public void delete() {
        if (r == f) {
            System.out.println("\nQueue is empty.");
            return;
        }
    }
}
```

```
System.out.println("\nProcessing and leaking packets...");
for (int i = f; i < r; i++) {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        System.out.println("Error: " + e.getMessage());
    }
    System.out.println("\nLeaked Packet: " + q[i]);
}
f = r = 0;
}
```

```
public static void main(String[] args) {
    Scanner src = new Scanner(System.in);
    System.out.print("\nEnter the queue size: ");
    int queueSize = src.nextInt();
    Queue q = new Queue(queueSize);
    System.out.print("\nEnter the number of packets to be sent: ");
    int size = src.nextInt();
    q.insert(size);
    q.delete();
}
}
```

OUTPUT:

Enter the queue size: **3**

Enter the number of packets to be sent: **5**

Enter element 1: **10**

Enter element 2: **20**

Enter element 3: **30**

Enter element 4: **40**

Queue is full. Lost Packet: **40**

Processing and leaking packets...

Leaked Packet: **10**

Leaked Packet: **20**

Leaked Packet: **30**

2. Implement simple ESS and with transmitting nodes in wire-less LAN by simulation and determine the throughput with respect to transmission of packets.

```
set ns [new Simulator]
set tf [open lab4.tr w]
$ns trace-all $tf
set topo [new Topography]
$topo load_flatgrid 1000 1000
set nf [open lab4.nam w]
$ns namtrace-all-wireless $nf 1000 1000

$ns node-config -adhocRouting DSDV \
    -llType LL \
    -macType Mac/802_11 \
    -ifqType Queue/DropTail \
    -ifqLen 50 \
    -phyType Phy/WirelessPhy \
    -channelType Channel/WirelessChannel \
    -propType Propagation/TwoRayGround \
    -antType Antenna/OmniAntenna \
    -topoInstance $topo \
    -agentTrace ON \
    -routerTrace ON

create-god 3
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
$n0 label "tcp0"
$n1 label "sink1/tcp1"
$n2 label "sink2"
$n0 set X_ 50
$n0 set Y_ 50
$n0 set Z_ 0
$n1 set X_ 100
$n1 set Y_ 100
$n1 set Z_ 0
$n2 set X_ 600
$n2 set Y_ 600
$n2 set Z_ 0
$ns at 0.1 "$n0 setdest 50 50 15"
$ns at 0.1 "$n1 setdest 100 100 25"
$ns at 0.1 "$n2 setdest 600 600 25"
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0
set sink1 [new Agent/TCPSink]
$ns attach-agent $n1 $sink1
$ns connect $tcp0 $sink1
set tcp1 [new Agent/TCP]
```

```
$ns attach-agent $n1 $tcp1
set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1
set sink2 [new Agent/TCPSink]
$ns attach-agent $n2 $sink2
$ns connect $tcp1 $sink2
$ns at 5 "$ftp0 start"
$ns at 5 "$ftp1 start"

$ns at 100 "$n1 setdest 550 550 15"
$ns at 190 "$n1 setdest 70 70 15"
proc finish { } {
    global ns nf tf
    $ns flush-trace
    exec nam lab4.nam &
    close $tf
    exit 0
}
$ns at 250 "finish"
$ns run
```

AWK file (Open a new editor using “vi command” and write awk file and save with “.awk” extension)

```
BEGIN{
    count1=0
    count2=0
    pack1=0
    pack2=0
    time1=0
    time2=0
}
{
    if($1=="r"&& $3=="_1_" && $4=="AGT")
    {
        count1++
        pack1=pack1+$8
        time1=$2
    }
    if($1=="r" && $3=="_2_" && $4=="AGT")
    {
        count2++
        pack2=pack2+$8
        time2=$2
    }
}
END{

printf("The Throughput from n0 to n1: %f Mbps \n", ((count1*pack1*8)/(time1*1000000)));
printf("The Throughput from n1 to n2: %f Mbps", ((count2*pack2*8)/(time2*1000000)));
}
```


Steps for execution

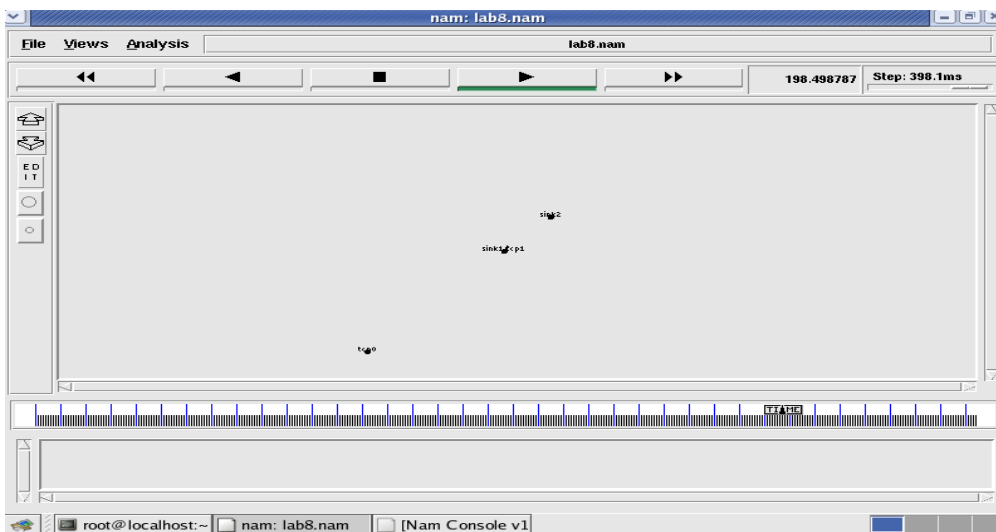
- 1) Open vi editor and type program. Program name should have the extension “.tcl”
`[root@localhost ~] # vi lab4.tcl`
- 2) Save the program by pressing “ESC key” first, followed by “Shift and :” keys simultaneously and type “wq” and press **Enter** key.
- 3) Open vi editor and type **awk** program. Program name should have the extension “awk”
`[root@localhost~] #vi lab4.awk`
- 4) Save the program by pressing “ESC key” first, followed by “Shift and:” keys simultaneously and type “wq” and press **Enter** key.
- 5) Run the simulation program `[root@localhost~] #ns lab4.tcl`
 - i) Here “ns” indicates network simulator. We get the topology shown in the snapshot.
 - ii) Now press the play button in the simulation window and the simulation will begin.
- 6) After simulation is completed run **awk file** to see the output,

```
[root@localhost~]# awk -f lab4.awk lab4.tr
```

- 7) To see the trace file contents open the file as,

```
[root@localhost~]# vi lab4.tr
```

Topology



Node 1 and 2 are communication

Trace file

Here “M” indicates mobile nodes, “AGT” indicates Agent Trace, “RTR” indicates Router Trace

```

root@localhost:~
File Edit View Terminal Tabs Help
s 0.036400876 _0_ RTR --- 0 message 32 [0 0 0 0] ----- [0:255 -1:255 32 0]
r 0.037421112 _1_ RTR --- 0 message 32 [0 ffffffff 0 800] ----- [0:255 -1:255
32 0]
M 0.10000 0 (50.00, 50.00, 0.00), (50.00, 50.00), 15.00
M 0.10000 1 (100.00, 100.00, 0.00), (100.00, 100.00), 25.00
M 0.10000 2 (600.00, 600.00, 0.00), (600.00, 600.00), 25.00
s 0.182633994 _1_ RTR --- 1 message 32 [0 0 0 0] ----- [1:255 -1:255 32 0]
r 0.183694230 _0_ RTR --- 1 message 32 [0 ffffffff 1 800] ----- [1:255 -1:255
32 0]
s 0.882774710 _2_ RTR --- 2 message 32 [0 0 0 0] ----- [2:255 -1:255 32 0]
s 5.000000000 _0_ AGT --- 3 tcp 40 [0 0 0 0] ----- [0:0 1:0 32 0] [0 0] 0 0
r 5.000000000 _0_ RTR --- 3 tcp 40 [0 0 0 0] ----- [0:0 1:0 32 0] [0 0] 0 0
s 5.000000000 _0_ RTR --- 3 tcp 60 [0 0 0 0] ----- [0:0 1:0 32 1] [0 0] 0 0
s 5.000000000 _1_ AGT --- 4 tcp 40 [0 0 0 0] ----- [1:1 2:0 32 0] [0 0] 0 0
r 5.000000000 _1_ RTR --- 4 tcp 40 [0 0 0 0] ----- [1:1 2:0 32 0] [0 0] 0 0
r 5.004812650 _1_ AGT --- 3 tcp 60 [13a 1 0 800] ----- [0:0 1:0 32 1] [0 0] 1
0
s 5.004812650 _1_ AGT --- 5 ack 40 [0 0 0 0] ----- [1:0 0:0 32 0] [0 0] 0 0
r 5.004812650 _1_ RTR --- 5 ack 40 [0 0 0 0] ----- [1:0 0:0 32 0] [0 0] 0 0
s 5.004812650 _1_ RTR --- 5 ack 60 [0 0 0 0] ----- [1:0 0:0 32 0] [0 0] 0 0
r 5.006977357 _0_ AGT --- 5 ack 60 [13a 0 1 800] ----- [1:0 0:0 32 0] [0 0] 1
0
s 5.006977357 _0_ AGT --- 6 tcp 1040 [0 0 0 0] ----- [0:0 1:0 32 0] [1 0] 0 0
"lab8.tr" 128664L, 11456314C 1,1 Top

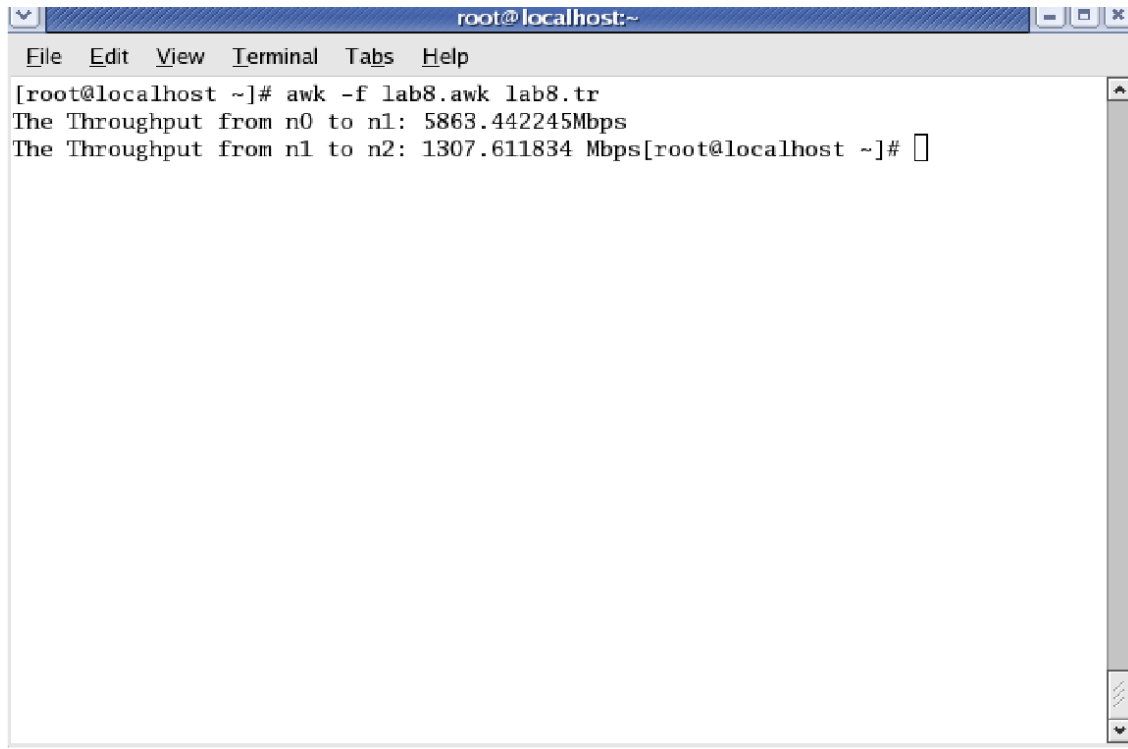
```

Output

```

Applications Actions Tue Dec 14, 3:30 PM
root@localhost:~
File Edit View Terminal Tabs Help
[root@localhost ~]# vi lab8.tcl
[root@localhost ~]# ns lab8.tcl
warning: Please use -channel as shown in tcl/ex/wireless-mitf.tcl
num_nodes is set 3
INITIALIZE THE LIST xListHead
channel.cc:sendUp - Calc highestAntennaZ_ and distCST_
highestAntennaZ_ = 1.5, distCST_ = 550.0
SORTING LISTS ...DONE!
[root@localhost ~]#

```



A terminal window titled 'root@localhost:~' with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal displays the output of an awk command: '[root@localhost ~]# awk -f lab8.awk lab8.tr'. The output shows two lines of throughput data: 'The Throughput from n0 to n1: 5863.442245Mbps' and 'The Throughput from n1 to n2: 1307.611834 Mbps'. The prompt '[root@localhost ~]#' is followed by a cursor.

```
root@localhost:~  
File Edit View Terminal Tabs Help  
[root@localhost ~]# awk -f lab8.awk lab8.tr  
The Throughput from n0 to n1: 5863.442245Mbps  
The Throughput from n1 to n2: 1307.611834 Mbps[root@localhost ~]#
```

3. Write a program for error detecting code using CRC CCITT (16-bits).

```
import java.io.*;

class Crc {

    public static void main(String args[]) throws IOException {

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int[] data;

        int[] div;

        int[] divisor;

        int[] rem;

        int[] crc;

        int data_bits, divisor_bits, tot_length;

        // Input: Number of data bits

        System.out.println("Enter number of data bits: ");

        data_bits = Integer.parseInt(br.readLine());

        data = new int[data_bits];

        // Input: Data bits

        System.out.println("Enter data bits: ");

        for (int i = 0; i < data_bits; i++) {

            data[i] = Integer.parseInt(br.readLine());

        }

        // Input: Number of bits in divisor

        System.out.println("Enter number of bits in divisor: ");

        divisor_bits = Integer.parseInt(br.readLine());

        divisor = new int[divisor_bits];

        // Input: Divisor bits

        System.out.println("Enter Divisor bits: ");

        for (int i = 0; i < divisor_bits; i++) {

            divisor[i] = Integer.parseInt(br.readLine());

        }

        // Calculate total length
```

```
tot_length = data_bits + divisor_bits - 1;

div = new int[tot_length];
rem = new int[tot_length];
crc = new int[tot_length];

// Append data bits to dividend
for (int i = 0; i < data.length; i++) {
    div[i] = data[i];}

// Print dividend after appending 0's
System.out.print("Dividend (after appending 0's): ");
for (int i = 0; i < div.length; i++) {
    System.out.print(div[i]);}
System.out.println();

// Copy dividend to remainder
for (int j = 0; j < div.length; j++) {
    rem[j] = div[j];
}

// Perform CRC division
for (int i = 0; i < data_bits; i++) {
    if (rem[i] == 1) {
        for (int j = 0; j < divisor_bits; j++) {
            rem[i + j] = rem[i + j] ^ divisor[j]; // XOR operation
        }
    }
}

// Extract CRC bits
for (int i = 0; i < crc.length; i++) {
    crc[i] = rem[i];
}

// Print CRC bits
System.out.print("CRC bits: ");
```

```
for (int i = data_bits; i < tot_length; i++) {  
    System.out.print(crc[i]);  
}  
System.out.println();  
// Transmit data with CRC  
System.out.print("Transmitted Data: ");  
for (int i = 0; i < data_bits; i++) {  
    System.out.print(data[i]);  
}  
for (int i = data_bits; i < tot_length; i++) {  
    System.out.print(crc[i]);  
}  
System.out.println();  
}
```

OUTPUT:

Enter number of data bits: **4**

Enter data bits:

1

0

0

1

Enter number of bits in divisor: **3**

Enter Divisor bits:

1

1

0

Dividend (after appending 0's): **100100**

CRC bits: **10**

Transmitted Data: **100110**

4 Implement transmission of ping messages/trace route over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion in the network.

```
set ns [ new Simulator ]
set nf [ open lab2.nam w ]
$ns namtrace-all $nf
set tf [ open lab2.tr w ]
$ns trace-all $tf
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
$n4 shape box
$ns duplex-link $n0 $n4 1005Mb 1ms DropTail
$ns duplex-link $n1 $n4 50Mb 1ms DropTail
$ns duplex-link $n2 $n4 2000Mb 1ms DropTail
$ns duplex-link $n3 $n4 200Mb 1ms DropTail
$ns duplex-link $n4 $n5 1Mb 1ms DropTail
set p1 [new Agent/Ping]
$ns attach-agent $n0 $p1
$p1 set packetSize_ 50000
$p1 set interval_ 0.0001
set p2 [new Agent/Ping]
$ns attach-agent $n1 $p2
set p3 [new Agent/Ping]
$ns attach-agent $n2 $p3
$p3 set packetSize_ 30000
$p3 set interval_ 0.00001
set p4 [new Agent/Ping]
$ns attach-agent $n3 $p4
set p5 [new Agent/Ping]
$ns attach-agent $n5 $p5
$ns queue-limit $n0 $n4 5
$ns queue-limit $n2 $n4 3
$ns queue-limit $n4 $n5 2
Agent/Ping instproc recv {from rtt} {
$self instvar node_
puts "node [$node_ id] received answer from $from with round trip time $rtt msec"
}
# please provide space between $node_ and id. No space between $ and from. No
#space between and $ and rtt */
```

```
$ns connect $p1 $p5
$ns connect $p3 $p4
proc finish { } {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam lab2.nam &
    exit 0
}
$ns at 0.1 "$p1 send"
$ns at 0.2 "$p1 send"
$ns at 0.3 "$p1 send"
$ns at 0.4 "$p1 send"
$ns at 0.5 "$p1 send"
$ns at 0.6 "$p1 send"
$ns at 0.7 "$p1 send"
$ns at 0.8 "$p1 send"
$ns at 0.9 "$p1 send"
$ns at 1.0 "$p1 send"
$ns at 1.1 "$p1 send"
$ns at 1.2 "$p1 send"
$ns at 1.3 "$p1 send"
$ns at 1.4 "$p1 send"
$ns at 1.5 "$p1 send"
$ns at 1.6 "$p1 send"
$ns at 1.7 "$p1 send"
$ns at 1.8 "$p1 send"
$ns at 1.9 "$p1 send"
$ns at 2.0 "$p1 send"
$ns at 2.1 "$p1 send"
$ns at 2.2 "$p1 send"
$ns at 2.3 "$p1 send"
$ns at 2.4 "$p1 send"
$ns at 2.5 "$p1 send"
$ns at 2.6 "$p1 send"
$ns at 2.7 "$p1 send"
$ns at 2.8 "$p1 send"
$ns at 2.9 "$p1 send"
$ns at 0.1 "$p3 send"
$ns at 0.2 "$p3 send"
$ns at 0.3 "$p3 send"
$ns at 0.4 "$p3 send"
$ns at 0.5 "$p3 send"
```



```
$ns at 0.6 "$p3 send"
$ns at 0.7 "$p3 send"
$ns at 0.8 "$p3 send"
$ns at 0.9 "$p3 send"
$ns at 1.0 "$p3 send"
$ns at 1.1 "$p3 send"
$ns at 1.2 "$p3 send"
$ns at 1.3 "$p3 send"
$ns at 1.4 "$p3 send"
$ns at 1.5 "$p3 send"
$ns at 1.6 "$p3 send"
$ns at 1.7 "$p3 send"
$ns at 1.8 "$p3 send"
$ns at 1.9 "$p3 send"
$ns at 2.0 "$p3 send"
$ns at 2.1 "$p3 send"
$ns at 2.2 "$p3 send"
$ns at 2.3 "$p3 send"
$ns at 2.4 "$p3 send"
$ns at 2.5 "$p3 send"
$ns at 2.6 "$p3 send"
$ns at 2.7 "$p3 send"
$ns at 2.8 "$p3 send"
$ns at 2.9 "$p3 send"
$ns at 3.0 "finish"
$ns run
```

AWK file (Open a new editor using “vi command” and write awk file and save with “.awk” extension)

```
BEGIN{
drop=0;
}
{
if($1= "d" )
{
drop++;
}
}
END{
printf("Total number of %s packets dropped due to congestion =%d\n",$5,drop);
}
```

Steps for execution

- 8) Open vi editor and type program. Program name should have the extension “.tcl”

```
[root@localhost ~] # vi lab2.tcl
```

- 9) Save the program by pressing “**ESC key**” first, followed by “**Shift and :**” keys simultaneously and type “**wq**” and press **Enterkey**.
- 10) Open vi editor and type **awk** program. Program name should have the extension “**.awk**”

```
[root@localhost ~] # vi lab2.awk
```

- 11) Save the program by pressing “**ESC key**” first, followed by “**Shift and :**” keys simultaneously and type “**wq**” and press **Enterkey**.
- 12) Run the simulationprogram

```
[root@localhost~] # ns lab2.tcl
```

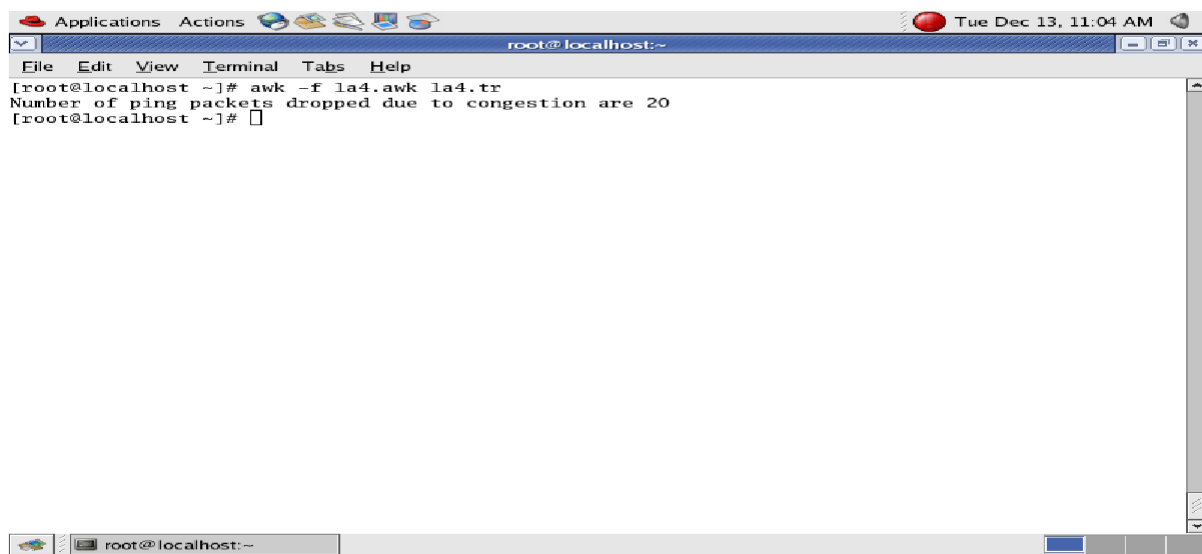
- i) Here “**ns**” indicates network simulator. We get the topology shown in the snapshot.
 - ii) Now press the play button in the simulation window and the simulation will begins.
- 13) After simulation is completed run **awk file** to see the output,

```
[root@localhost~]# awk -f lab2.awklab2.tr
```

- 14) To see the trace file contents open the file as,

```
[root@localhost~]# vi lab2.tr
```

Topology



Applications Actions Tue Dec 13 10:41 AM

5. Write a program to find the shortest path between vertices using bellman-ford algorithm.

```
import java.util.Scanner;
public class BellmanFord
{
    private int D[];
    private int num_ver;
    public static final int MAX_VALUE = 999;
    public BellmanFord(int num_ver)
    {
        this.num_ver = num_ver;
        D = new int[num_ver + 1];
    }
    public void BellmanFordEvaluation(int source, int A[][])
    {
        for (int node = 1; node <= num_ver; node++)
        {
            D[node] = MAX_VALUE;
        }
        D[source] = 0;
        for (int node = 1; node <= num_ver - 1; node++)
        {
            for (int sn = 1; sn <= num_ver; sn++)
            {
                for (int dn = 1; dn <= num_ver; dn++)
                {
                    if (A[sn][dn] != MAX_VALUE)
                    {
                        if (D[dn] > D[sn] + A[sn][dn])
                            D[dn] = D[sn] + A[sn][dn];
                    }
                }
            }
        }
        for (int sn = 1; sn <= num_ver; sn++)
        {
            for (int dn = 1; dn <= num_ver; dn++)
            {
                if (A[sn][dn] != MAX_VALUE)
                {
                    if (D[dn] > D[sn] + A[sn][dn])
                        System.out.println("The Graph contains negative egde cycle");
                }
            }
        }
        for (int vertex = 1; vertex <= num_ver; vertex++)
        {
            System.out.println("distance of source " + source + " to " +
                               vertex + " is " + D[vertex]);
        }
    }
}
```

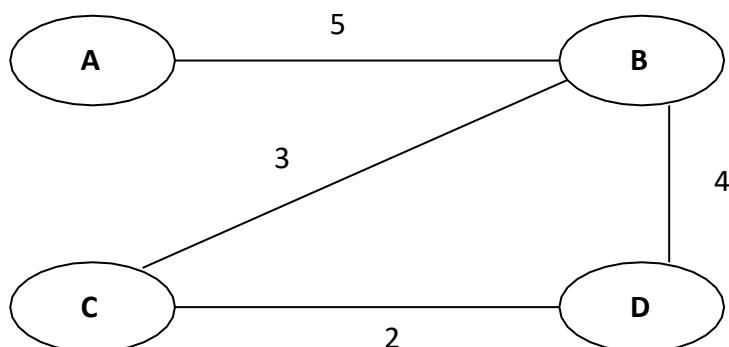
```

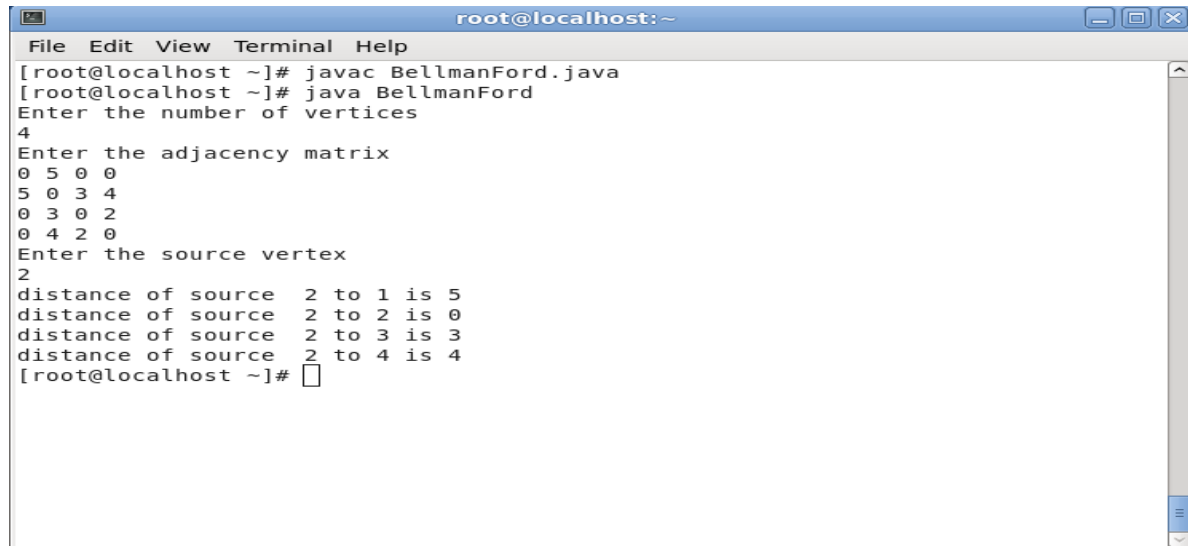
    }
}
public static void main(String[ ] args)
{
    int
    num_ver
    = 0; int
    source;
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the number of
    vertices"); num_ver = scanner.nextInt();
    int A[][] = new int[num_ver + 1][num_ver
    + 1]; System.out.println("Enter the
    adjacency matrix"); for (int sn = 1; sn <=
    num_ver; sn++)
    {
        for (int dn = 1; dn <= num_ver; dn++)
        {
            A[sn][dn] = scanner.nextInt(); if (sn == dn)
            {
                A[sn][dn] = 0;
                continue;
            }
            if (A[sn][dn] == 0)
            {
                A[sn][dn] = MAX_VALUE;
            }
        }
    }
    System.out.println("Enter the source vertex");
    source = scanner.nextInt();
    BellmanFord b = new BellmanFord (num_ver);
    b.BellmanFordEvaluation(source, A);
    scanner.close();
}
}

```

INPUT:

[root@localhost ~]# vi BellmanFord.java



OUTPUT:

```
root@localhost:~  
File Edit View Terminal Help  
[root@localhost ~]# javac BellmanFord.java  
[root@localhost ~]# java BellmanFord  
Enter the number of vertices  
4  
Enter the adjacency matrix  
0 5 0 0  
5 0 3 4  
0 3 0 2  
0 4 2 0  
Enter the source vertex  
2  
distance of source 2 to 1 is 5  
distance of source 2 to 2 is 0  
distance of source 2 to 3 is 3  
distance of source 2 to 4 is 4  
[root@localhost ~]#
```

6. Implement an Ethernet LAN using n nodes and set multiple traffic nodes and plot congestion window for different source / destination.

```
set ns [new Simulator]
set tf [open lab3.tr w]
$ns trace-all $tf
set nf [open lab3.nam w]
$ns namtrace-all $nf
set n0 [$ns node]
$n0 color "magenta"
$n0 label "src1"
set n1 [$ns node]
set n2 [$ns node]
$n2 color "magenta"
$n2 label "src2"
set n3 [$ns node]
$n3 color "blue"
$n3 label "dest2"
set n4 [$ns node]
set n5 [$ns node]
$n5 color "blue"
$n5 l
abel "dest1"

$ns make-lan "$n0 $n1 $n2 $n3 $n4" 100Mb 100ms LL Queue/DropTail Mac/802_3

/* should come in single line */

$ns duplex-link $n4 $n5 1Mb 1ms DropTail

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0
$ftp0 set packetSize_ 500
$ftp0 set interval_ 0.0001
set sink5 [new Agent/TCPSink]
$ns attach-agent $n5 $sink5
$ns connect $tcp0
$sink5 set tcp2 [new Agent/TCP]
$ns attach-agent $n2 $tcp2
set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2
$ftp2 set packetSize_ 600
$ftp2 set interval_ 0.001
set sink3 [new Agent/TCPSink]
$ns attach-agent $n3 $sink3
$ns connect $tcp2 $sink3
set file1 [open file1.tr w]
$tcp0 attach $file1
set file2 [open file2.tr w]
```

```
$tcp2 attach $file2
$tcp0 trace cwnd_ /* must put underscore ( _ ) after cwnd and no space between
them*/

$tcp2 trace cwnd_
proc finish { } {
    global ns nf tf
    $ns flush-trace
    close $tf
    close $nf
    exec nam lab3.nam &
    exit 0
}
$ns at 0.1 "$ftp0 start"
$ns at 5 "$ftp0 stop"
$ns at 7 "$ftp0 start"
$ns at 0.2 "$ftp2 start"
$ns at 8 "$ftp2 stop"
$ns at 14 "$ftp0 stop"
$ns at 10 "$ftp2 start"
$ns at 15 "$ftp2 stop"
$ns at 16 "finish"
$ns run
```

AWK file (Open a new editor using “vi command” and write awk file and save with “.awk” extension)

cwnd:- means congestion window

```
BEGIN
{
{
if($6= "cwnd_") /* don't leave space after writing cwnd_ */
printf("%ft%ft\n",$1,$7); /* you must put \n in printf */
}
END
{
}
```

Steps for execution

- 15) Open vi editor and type program. Program name should have the extension “.tcl”

[root@localhost ~]# vi lab3.tcl

- 16) Save the program by pressing “**ESC key**” first, followed by “**Shift and :**” keys simultaneously and type “**wq**” and press **Enterkey**.
- 17) Open vi editor and type **awk** program. Program name should have the extension “.awk”


```
[root@localhost ~] # vi lab3.awk
```

- 18) Save the program by pressing “**ESC key**” first, followed by “**Shift and:**” keys simultaneously and type “**wq**” and press **Enterkey**.
- 19) Run the simulation program

```
[root@localhost~] # ns lab3.tcl
```

- 20) After simulation is completed run **awk file** to see the output,

i. [root@localhost~] # awk -f lab3.awk file1.tr >a1

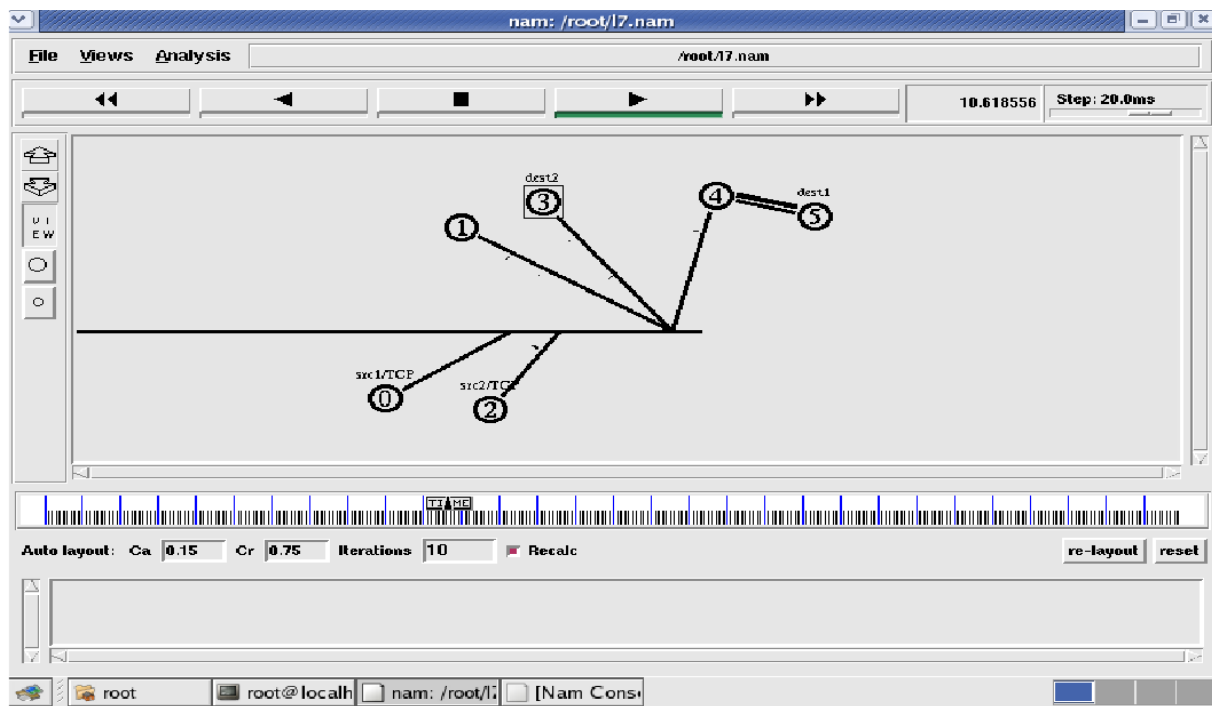
ii. [root@localhost~] # awk -f lab3.awk file2.tr >a2

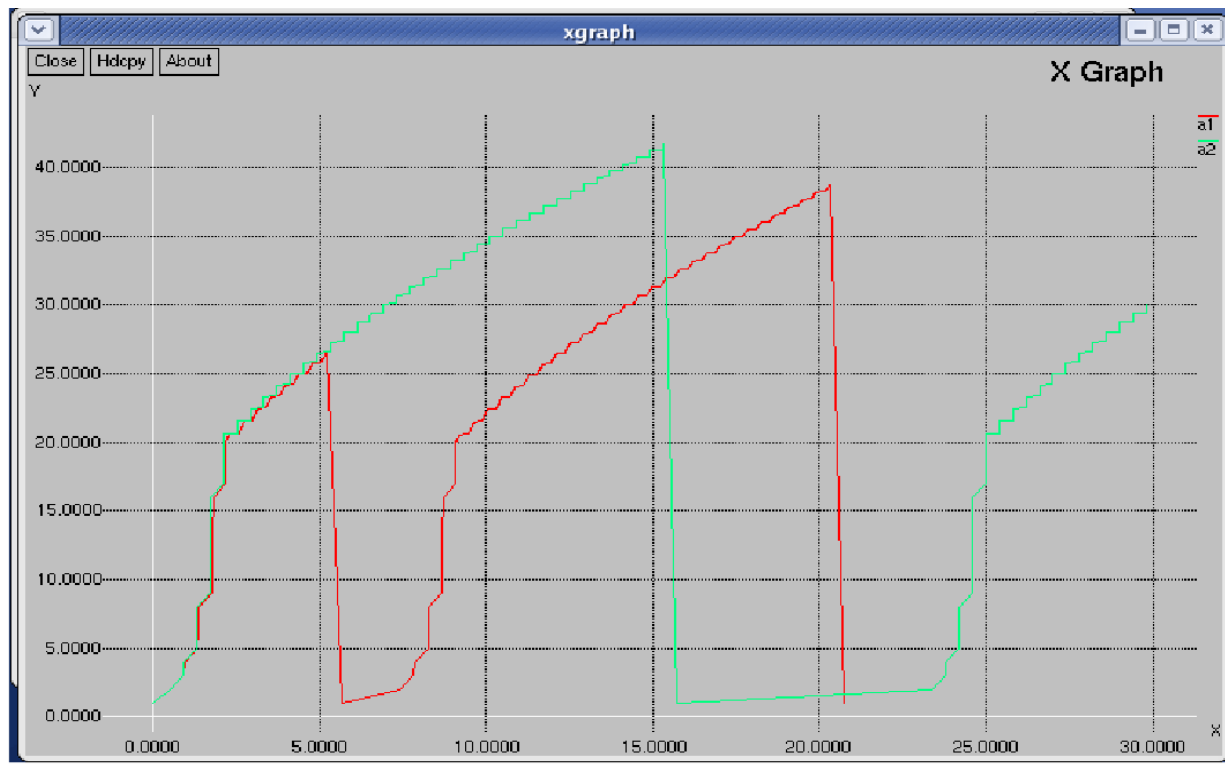
iii. [root@localhost~] # xgraph a1a2

- 21) Here we are using the congestion window trace files i.e. **file1.tr** and **file2.tr** and we are redirecting the contents of those files to new files say **a1** and **a2** using **output redirection operator (>)**.
- 22) To see the trace file contents open the file as,

```
[root@localhost~]# vi lab3.tr
```

Topology



OUTPUT:

7. Write a program for congestion control using leaky bucket algorithm.

```
import java.io.*; import
java.util.*; class Queue
{
    int q[],f=0,r=0,size;
    void insert(int n)
    {
        Scanner in = new Scanner(System.in);
        q=new int[10];
        for(int i=0;i<n;i++)
        {
            System.out.print("\nEnter " + i + " element: ");
            int ele=in.nextInt();
            if(r+1>10)
            {
                System.out.println("\nQueue is full \nLost Packet: "+ele);
                break;
            }
            else
            {
                r++;
                q[i]=ele;
            }
        }
    }

    void delete()
    {
        Scanner in = new Scanner(System.in);
        Thread t=new Thread();
        if(r==0)
            System.out.print("\nQueue empty ");
        else
        {
            for(int i=f;i<r;i++)
            {
                try
                {
                    t.sleep(1000);
                }
                catch(Exception e){}
                System.out.print("\nLeaked Packet: "+q[i]);
                f++;
            }
        }
        System.out.println();
    }
}

class Leaky extends Thread
```

```
{  
    public static void main(String ar[]) throws Exception  
    {  
        Queue q=new Queue();  
        Scanner src=new Scanner(System.in);  
        System.out.println("\nEnter the packets to be sent:");  
        int size=src.nextInt();  
        q.insert(size);  
        q.delete();  
    }  
}
```

OUTPUT:

```
File Edit View Terminal Help  
  
Enter the packets to be sent:  
12  
  
Enter 0 element: 2  
Enter 1 element: 3  
Enter 2 element: 5  
Enter 3 element: 6  
Enter 4 element: 8  
Enter 5 element: 9  
Enter 6 element: 4  
Enter 7 element: 5  
Enter 8 element: 6  
Enter 9 element: 2  
Enter 10 element: 3  
  
Queue is full  
Lost Packet: 3
```

```
Leaked Packet: 2  
Leaked Packet: 3  
Leaked Packet: 5  
Leaked Packet: 6  
Leaked Packet: 8  
Leaked Packet: 9  
Leaked Packet: 4
```